Take Home Test Report

Krishnan Raghavan

This solution is implemented in pure-python without any external dependencies using the built-in **argparse** package as a CLI interface. It exposes various CLI parameters and provides options to customise output. Run the below to see detailed documentation.

```
$ ./package_statistics.py -h
usage: package_statistics.py [-h] [-r REPOSITORY_URL] [--code-name CODE_NAME] [--component
COMPONENT] [--count COUNT] [--udeb] [-v] architecture
...
```

Implementation

The code is logically separated into different concerns, *the CLI interface*, *HTTP fetching logic*, *statistics calculation logic* and *output formatting* each split into a separate function. The implementation uses the Python IO stream abstraction to process the entire index without loading the whole file into memory from the network.

Code & Tooling

The code was auto-formatted using the **black** library. **flake8** was used to run multiple linters including automated checks for the PEP8 style guide. **mypy** was used to type check code with included annotations.

Development Effort

The task in total required almost ~4-6 hours of development effort. The first step was understanding the problem and how Debian repositories work, the different indices, their file format and the wire protocol of the API over the network. Subsequently I spent time evaluating different implementation options, including different approaches like using concurrent async I/O, blocking thread pools etc and different options for libraries and data structures to use for the implementation. I have also made reasonable assumptions where there were ambiguities in the specification (like the format of the index header), which in practice could be clarified with further discussion at the requirement gathering stage.

Future TODOs

The following are various concerns not addressed in this implementation to keep the scope and effort involved reasonable:

- **Packaging**: Although Python packaging is complex, distributing this script would involve adding packaging metadata including a binary symlink using setuptools to the prefix ‘bin’ path.
- **Unit testing:** My tool of choice would be **pytest** to run tests. The implementation also splits the code into different functions which can be tested independently or refactored into separate files in future.
- **Backward Compatibility:** This code uses various Python 3.10+ features including type annotations that could be made backward compatible with ‘future’ imports and automated tests for different versions using a test automation tool like **tox**
- **Error Handling** & **Input Validation** is not very sophisticated, including sanitising untrusted user input sent to the network if the script is embedded in other places.
- **Output Formatting:** Fetching indices is a time taking operation, for interactive usage progress bars can be rendered with libraries like **rich**. Support for JSON & YAML output through a flag for scripting uses would also be nice to have.
- Better index **file downloading** logic, including caching and resuming downloads.